HTML NOTES

🌐 HTML — COMPLETE QUICK REVISION NOTES

Beginner → Professional Foundation

```
HTML
 ↓
Structure
 ↓
Semantic meaning
 ↓
Forms
 ↓
Accessibility
 ↓
SEO
 ↓
Production HTML
 ↓
React/Next.js foundation
```

🟢 LEVEL 1 — ABSOLUTE BEGINNER

1. What is HTML?

HTML = HyperText Markup Language

It defines the **structure and meaning** of a web page.

```
HTML → Structure
CSS → Appearance
JavaScript → Behavior
```

Example:

```
<h1>Hello World</h1>
<p>This is my website.</p>
```

---

2. Basic HTML document

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <title>My Website</title>
</head>

<body>

    <h1>Hello</h1>

</body>

</html>
```

Important parts

Element	Purpose
<!DOCTYPE html>	Declares modern HTML
<html>	Root element
<head>	Metadata
<body>	Visible page content
<title>	Browser/tab title
<meta>	Metadata

---

3. Elements

```
<p>Hello</p>
```

Usually:

```
Opening tag
     ↓
<p>
     ↓
Content
     ↓
</p>
     ↓
Closing tag
```

Some elements are **void elements**:

<img>
<input>
**
**
<hr>
<meta>
<link>

They don't have normal closing tags.

LEVEL 2 — CORE HTML

4. Headings

```
<h1>Main heading</h1>
<h2>Section</h2>
<h3>Subsection</h3>
```

Think:

```
h1
 ↓
Main topic

h2
 ↓
Major section

h3
 ↓
Subsection
```

Use headings for **structure**, not merely font size.

---

5. Paragraph

```
<p>This is a paragraph.</p>
```

---

6. Links

```
<a href="https://example.com">
    Visit website
</a>
```

Internal:

```
<a href="/pricing">Pricing</a>
```

Remember

```
<a>
 ↓
Navigation
 ↓
Go somewhere
```

---

7. Images

```
<img
    src="image.jpg"
    alt="AI dashboard"
>
```

Important attributes:

src → image location
alt → alternative text

---

8. Lists

Unordered

```
<ul>
    <li>React</li>
    <li>Python</li>
</ul>
```

Ordered

```
<ol>
    <li>Login</li>
    <li>Upload PDF</li>
</ol>
```

Description list

```
<dl>
    <dt>HTML</dt>
    <dd>Web page structure</dd>
</dl>
```

---

9. Attributes

```
<input
    type="email"
    id="email"
    name="email"
    required
>
```

Attributes provide additional information/configuration.

Common:

id
class
src
href
alt
type

name
value
placeholder
required
disabled
readonly

---

10. `id` VS `class`

id	class
Usually identifies one specific element	Reusable grouping
Should be unique in a document	Can be reused
Useful for labels/anchors/JS	Very common for CSS/JS

Example:

```
<input id="email" class="input">
```

LEVEL 3 — INTERMEDIATE

11. Semantic HTML

Semantic HTML tells the browser **what something means.**

Important elements:

<header>
<nav>
<main>
<section>
<article>
<aside>
<footer>

Example:

```
<header>
    <nav>...</nav>
</header>

<main>
    <section>...</section>
    <article>...</article>
    <aside>...</aside>
</main>

<footer>...</footer>
```

Core idea

```
<div>
  ↓
Generic container

Semantic element
  ↓
Meaningful structure
```

---

12. `<div>` vs semantic elements

❌

```
<div>Navigation</div>
<div>Main content</div>
<div>Footer</div>
```

Better:

```
<nav>Navigation</nav>
<main>Main content</main>
<footer>Footer</footer>
```

Use `<div>` when no more appropriate semantic element exists.

---

13. Forms

Basic:

```
<form>

    <label for="email">
        Email
    </label>

    <input
        id="email"
        name="email"
        type="email"
    >

    <button type="submit">
        Submit
    </button>

</form>
```

---

14. Important form elements

```
<form>
<label>
<input>
<textarea>
<select>
<option>
<button>
```

---

15. Input types

Important types:

```
<input type="text">
<input type="email">
<input type="password">
<input type="number">
<input type="date">
<input type="file">
<input type="checkbox">
<input type="radio">
<input type="url">
<input type="search">
```

---

16. Form validation

HTML can perform basic client-side validation.

```
<input
    type="email"
    required
>
```

Other useful attributes:

```
required
minlength
maxlength
min
max
pattern
type
```

VERY IMPORTANT

```
Frontend validation
       ≠
Security
```

Backend must validate again.

---

17. `button` types

Inside forms:

```
<button type="submit">
    Submit
</button>

<button type="button">
    Generate
</button>

<button type="reset">
    Reset
</button>
```

Important:

A `<button>` inside a form can behave as a submit button by default, so explicitly choosing the correct `type` is good practice.

---

18. Link vs button

Link

```
<a href="/pricing">
    Pricing
</a>
```

Means:

Go somewhere.

Button

```
<button type="button">
    Generate
</button>
```

Means:

Perform an action.

Remember:

```
Navigation → <a>
Action → <button>
```

---

19. Tables

Use tables for **genuinely tabular data**.

```
<table>

    <thead>
        <tr>
            <th>Name</th>
            <th>Status</th>
        </tr>
    </thead>

    <tbody>
        <tr>
            <td>Project A</td>
            <td>Active</td>
        </tr>
    </tbody>

</table>
```

Important:

```
table
  ├── thead
  ├── tbody
  ├── tr
  ├── th
  └── td
```

Don't use tables for page layout.

🔴 LEVEL 4 — ADVANCED

20. Accessibility

Goal:

Make applications usable by as many people as reasonably possible, including users relying on assistive technologies.

Important:

Semantic HTML
Labels
Alt text

Keyboard access
Correct headings
Accessible names
Focus behavior
Useful errors

---

21. Labels

Good:

```
<label for="email">
    Email
</label>

<input id="email" type="email">
```

Connection:

```
label for="email"
      ↓
input id="email"
```

---

22. Image `alt`

Meaningful:

```
<img
    src="dashboard.png"
    alt="AI Email Writer dashboard"
>
```

Decorative:

```
<img
    src="decoration.png"
    alt=""
>
```

Don't put meaningless descriptions into `alt`.

---

23. ARIA

ARIA provides additional accessibility semantics.

Example:

```
<button aria-label="Close">
```

```
    X
</button>
```

Professional rule

Use native HTML whenever possible; use ARIA when necessary.

Don't replace:

```
<button>
```

with:

```
<div role="button">
```

unless there is a genuinely justified reason.

---

24. `figure` and `figcaption`

```
<figure>

    <img
        src="dashboard.png"
        alt="AI dashboard"
    >

    <figcaption>
        AI dashboard preview
    </figcaption>

</figure>
```

---

25. `<details>` / `<summary>`

Native expandable content:

```
<details>

    <summary>
        What is RAG?
    </summary>

    <p>
        Retrieval-Augmented Generation...
    </p>

</details>
```

Useful for:

FAQs
Documentation
Help
Explanations

---

26. Media

Audio

```
<audio controls>
    <source src="audio.mp3" type="audio/mpeg">
</audio>
```

Video

```
<video controls>
    <source src="demo.mp4" type="video/mp4">
</video>
```

---

27. `<iframe>`

Embeds another document/resource.

```
<iframe
    src="..."
    title="Embedded content">
</iframe>
```

Possible uses:

```
Maps
Videos
External widgets
Third-party content
```

---

28. Responsive foundation

Important viewport metadata:

```
<meta
    name="viewport"
    content="width=device-width, initial-scale=1"
>
```

This is especially important for mobile web pages.

---

29. Metadata

Example:

```
<head>

    <meta charset="UTF-8">

    <meta
        name="description"
        content="AI-powered PDF explainer"
    >

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1"
    >

    <title>AI PDF Explainer</title>

</head>
```

---

30. SEO basics

HTML helps search engines understand content.

Important:

```
<title>
<meta description>
<h1>
<h2>
Semantic HTML
Meaningful links
Image alt text
Logical content structure
```

🔴 LEVEL 5 — PROFESSIONAL FOUNDATION

31. Professional HTML mindset

Don't ask:

"Which tag looks right?"

Ask:

"What does this content/control actually mean?"

Then choose the appropriate element.

---

32. HTML + CSS + JS

Remember:

```
HTML
 ↓
Structure + meaning

CSS
 ↓
Appearance + layout

JavaScript
 ↓
Behavior + interaction
```

Example:

```
<button>
    Generate Email
</button>
```

HTML:

This is a button.

CSS:

Make it beautiful.

JavaScript:

When clicked, perform an action.

---

33. HTML + Backend

HTML doesn't provide real backend security or business logic.

architecture:

```
Browser
   ↓
HTML / React
   ↓
JavaScript
   ↓
HTTP
   ↓
FastAPI
   ↓
Database / AI / Services
```

---

34. Frontend validation vs backend validation

```
User
 ↓
Frontend validation
 ↓
HTTP request
 ↓
Backend validation
 ↓
Business logic
 ↓
Database
```

Never trust data simply because it came from your frontend.

---

35. Hidden ≠ secure

This:

```
<input
    type="hidden"
    name="role"
    value="admin"
>
```

doesn't make the user an admin.

Users can inspect/modify client-side data.

Security belongs on the server.

---

36. `disabled` vs `readonly`

disabled	readonly
User can't normally interact with it	User can't normally edit it
Different form behavior	Still represents a usable form control
Used when control isn't currently available	Used when value should be visible but not editable

---

37. hidden

```
<div hidden>
    Content
</div>
```

Means it isn't normally rendered.

But:

```
hidden
≠
secret
```

---

38. autocomplete

Useful for improving form usability:

```
<input
    type="email"
    autocomplete="email"
>
```

Examples:

name
email
username
current-password
new-password

---

39. Performance basics

Example:

```
<img
    src="large-image.jpg"
    alt="Dashboard"
    loading="lazy"
>
```

Useful for images that don't need to load immediately.

Image dimensions can also help layout stability:

```
<img
    src="dashboard.png"
    alt="Dashboard"
    width="1200"
    height="800"
>
```

Don't blindly optimize everything; understand the performance trade-off.

---

40. React connection

HTML:

```
<label for="email">
    Email
</label>
```

React JSX:

```
<label htmlFor="email">
    Email
</label>
```

The underlying concepts remain the same.

Your HTML knowledge directly helps with React.

🧠 MOST IMPORTANT DIFFERENCES

Concept	Remember
HTML vs CSS	Structure vs appearance
HTML vs JS	Structure vs behavior
`<a>` vs `<button>`	Navigation vs action
`<div>` vs semantic element	Generic vs meaningful
`id` vs `class`	Specific identifier vs reusable grouping
`disabled` vs `readonly`	Unavailable vs non-editable
`alt` vs visible text	Image alternative vs normal content
Frontend validation vs backend	User experience vs trusted validation
Hidden vs secure	Hidden from normal display ≠ protected
HTML vs React	Document markup vs component-based UI

🎯 HTML INTERVIEW CHEAT SHEET

Q1. What is HTML?

HTML is a markup language used to define the structure and semantic meaning of web content.

Q2. HTML vs CSS?

```
HTML → structure
CSS → presentation
```

Q3. What is semantic HTML?

Using elements according to their meaning, such as:

```
<header>
<nav>
<main>
<article>
<section>
<footer>
```

Q4. Why use semantic HTML?

It improves:

```
Accessibility
Maintainability
SEO
Code understanding
```

Q5. `<div>` vs `<section>`?

`div` is generic; `section` represents a meaningful thematic section of content.

Q6. `<a>` vs `<button>`?

```
<a> → navigation
<button> → action
```

Q7. What is `alt`?

Alternative text describing the useful meaning of an image.

Q8. Why use `<label>`?

It associates descriptive text with a form control and improves usability/accessibility.

Q9. Is HTML validation enough for security?

No. Backend validation is still required.

Q10. Is hidden input secure?

No. Client-side data must not be trusted.

Q11. What is ARIA?

Accessibility attributes that provide additional semantics when native HTML isn't sufficient.

Q12. What is the viewport meta tag?

It helps browsers appropriately size/layout pages on different device viewports.

Q13. What is semantic HTML's biggest advantage?

It gives content and controls **meaning**, rather than merely creating visual containers.

Q14. Can HTML itself call an AI model?

Not by itself. Application logic typically uses JavaScript and/or a backend/API to communicate with an AI service.

Q15. Why learn HTML before React?

Because React's UI is built on the same fundamental concepts of elements, attributes, forms, semantics, and accessibility.

✅ HTML COMPLETION CHECKPOINT

- ✅ LEVEL 1 — Absolute Beginner
- ✅ LEVEL 2 — Core HTML
- ✅ LEVEL 3 — Intermediate
- ✅ LEVEL 4 — Advanced
- ✅ LEVEL 5 — Professional Foundation

You should now be able to:

- ✅ Create HTML documents
- ✅ Structure pages
- ✅ Create forms
- ✅ Build navigation
- ✅ Use semantic HTML
- ✅ Build accessible interfaces

- Understand SEO fundamentals
- Understand validation
- Understand HTML security boundaries
- Structure real application interfaces
- Think in UI components
- Understand HTML → React connection
